

UniApp【萌宠案例】异步操作then-catch-finally严格处理方案

在萌宠案例的开发中，异步操作（如API请求、本地存储读写、图片保存等）广泛存在，若回调结果处理不严格，易出现数据异常、页面卡死、用户无感知错误等问题。本方案针对案例中核心异步场景，通过规范then-catch-finally使用，实现“成功结果校验、异常分类处理、最终状态重置”的完整链路，同时保持代码可读性与可维护性。

一、核心原则：then-catch-finally使用规范

异步操作处理需遵循“严格校验、全面捕获、必有收尾”的原则，三个环节各司其职：

- then回调**：不仅接收成功结果，更要对结果格式、核心字段进行严格校验，拒绝“信任式接收”，避免因后端数据异常导致前端崩溃。
- catch回调**：分类捕获不同类型异常（网络错误、权限拒绝、业务错误等），给出精准的用户提示，同时记录错误日志便于排查。
- finally回调**：负责清理操作，如关闭加载动画、重置请求状态，确保无论成功失败，页面状态都能回归正常，避免阻塞后续操作。



注意：UniApp中部分原生API（如uni.showToast）虽为异步，但未返回Promise，需通过回调函数手动封装Promise，再应用then-catch-finally规范。

二、基础封装：Promise化原生API

先对UniApp原生异步API进行Promise封装，统一调用范式，为后续严格处理奠定基础。

1. 通用工具类（utils/asyncUtil.js）

Code block

```
1  /**
2   * 异步操作工具类
3   * 1. Promise化UniApp原生异步API
4   * 2. 提供通用的异常分类方法
5   */
```

```

6 export const asyncUtil = {
7   /**
8    * Promise化UniApp API
9    * @param {Function} api - UniApp原生API (如uni.getStorage,
    uni.saveImageToPhotosAlbum)
10   * @param {Object} options - API参数
11   * @returns {Promise} - 包装后的Promise对象
12   */
13   promisify(api, options = {}) {
14     return new Promise((resolve, reject) => {
15       // 校验API合法性
16       if (typeof api !== 'function') {
17         reject(new Error(`API类型错误: ${api}不是函数`))
18         return
19       }
20       api({
21         ...options,
22         success: (res) => resolve(res),
23         fail: (err) => reject(err)
24       })
25     })
26   },
27
28   /**
29    * 异常分类处理
30    * @param {Error} err - 捕获的异常对象
31    * @returns {Object} - 分类后的异常信息
32    */
33   classifyError(err) {
34     const errMsg = err.message || '未知错误'
35     // 网络错误
36     if (errMsg.includes('网络') || errMsg.includes('timeout') || err.errCode
=== 1001) {
37       return { type: 'network', msg: '网络异常, 请检查网络连接', detail: err }
38     }
39     // 权限错误
40     if (errMsg.includes('auth') || errMsg.includes('权限')) {
41       return { type: 'permission', msg: '权限已拒绝, 请在设置中开启', detail: err
}
42     }
43     // 业务错误 (从API错误信息中提取)
44     if (errMsg.includes('业务') || err.code) {
45       return { type: 'business', msg: errMsg, detail: err }
46     }
47     // 其他错误
48     return { type: 'other', msg: '操作失败, 请重试', detail: err }
49   }

```

```

50 }
51
52 // 常用API的Promise化快捷方法
53 export const uniAsync = {
54   getStorage: (key) => asyncUtil.promisify(uni.getStorage, { key }),
55   setStorage: (key, data) => asyncUtil.promisify(uni.setStorage, { key, data
56   }),
57   removeStorage: (key) => asyncUtil.promisify(uni.removeStorage, { key }),
58   saveImageToPhotosAlbum: (filePath) =>
59   asyncUtil.promisify(uni.saveImageToPhotosAlbum, { filePath }),
60   openSetting: () => asyncUtil.promisify(uni.openSetting),
61   showModal: (options) => asyncUtil.promisify(uni.showModal, options)
62 }

```

2. 强化请求工具类（utils/request.js）

在原有请求工具类基础上，完善then-catch链路，确保API请求的全场景覆盖。

Code block

```

1  import { asyncUtil } from './asyncUtil'
2
3  // 基础配置
4  const baseConfig = {
5    baseUrl: 'https://api.pet.com',
6    timeout: 10000,
7    headers: {
8      'Content-Type': 'application/json',
9      'access-key': '' // 实际开发中从存储获取
10   }
11 }
12
13 /**
14  * 强化版请求工具类
15  * 自带then-catch基础处理，返回Promise供业务层二次处理
16  */
17 const request = (options) => {
18   // 1. 基础参数合并
19   const requestOptions = {
20     url: baseConfig.baseUrl + (options.url || ''),
21     method: options.method || 'GET',
22     header: { ...baseConfig.headers, ...options.header },
23     data: options.data || {},
24     timeout: options.timeout || baseConfig.timeout,
25     sslVerify: options.sslVerify !== false
26   }
27

```

```

28 // 2. 发起请求并返回Promise
29 return asyncUtil.promisify(uni.request, requestOptions)
30   .then((res) => {
31     const { statusCode, data } = res
32     // HTTP状态码校验
33     if (statusCode < 200 || statusCode >= 300) {
34       throw new Error(`HTTP错误: ${statusCode}, ${data.msg || '请求失败'}`)
35     }
36     // 业务状态码校验 (假设code=0为成功)
37     if (data.code !== 0) {
38       throw new Error(`业务错误: ${data.code}, ${data.msg || '操作异常'}`)
39     }
40     // 返回纯净业务数据
41     return data.data
42   })
43   .catch((err) => {
44     // 统一异常分类
45     const errorInfo = asyncUtil.classifyError(err)
46     // 错误日志上报 (实际开发中对接日志系统)
47     console.error(`[请求异常][${errorInfo.type}]`, errorInfo.detail)
48     // 抛出分类后的异常, 供业务层二次处理
49     throw errorInfo
50   })
51 }
52
53 // 简化GET/POST调用
54 request.get = (url, data = {}, options = {}) => {
55   return request({ url, method: 'GET', data, ...options })
56 }
57
58 request.post = (url, data = {}, options = {}) => {
59   return request({ url, method: 'POST', data, ...options })
60 }
61
62 export default request

```

三、核心场景：异步操作严格处理实现

针对萌宠案例中首页、详情页、工具类等核心场景，逐一实现规范的异步处理逻辑。

1. 首页：API请求与数据校验

首页核心异步操作为获取轮播图与推荐列表数据，需在then中严格校验数组格式，catch中分类提示，finally中关闭加载。

```
1 <template>
2   <view class="index-page">
3
4     <view class="loading-overlay" v-if="isLoading">加载中...</view>
5
6     <swiper v-if="swiperList.length" indicator-dots autoplay>
7       <swiper-item v-for="item in swiperList" :key="item.id">
8         <image :src="item.imgUrl" mode="widthFix"
@tap="previewImage(swiperList, index)"></image>
9       </swiper-item>
10    </swiper>
11
12    <view class="recommend-list" v-if="recommendList.length">
13
14    </view>
15
16    <view class="error-tip" v-if="errorMsg" @click="reloadData">{{ errorMsg }}
(点击重试) </view>
17  </view>
18 </template>
19
20 <script setup>
21 import { ref } from 'vue'
22 import request from '../utils/request'
23 import { imageUtil } from '../utils/imageUtil'
24
25 // 页面状态
26 const isLoading = ref(true)
27 const swiperList = ref([])
28 const recommendList = ref([])
29 const errorMsg = ref('')
30
31 // 加载首页数据
32 const loadHomeData = () => {
33   isLoading.value = true
34   errorMsg.value = '' // 重置错误信息
35   request.get('/home')
36     .then((homeData) => {
37     // 严格校验数据格式
38     if (typeof homeData !== 'object' || homeData === null) {
39       throw new Error('首页数据格式异常')
40     }
41     // 轮播图数据校验（必须为数组，且包含核心字段）
42     if (Array.isArray(homeData.swiper)) {
43       swiperList.value = homeData.swiper.filter(item => item.id &&
item.imgUrl)
44     } else {
```

```

45         throw new Error('轮播图数据缺失')
46     }
47     // 推荐列表数据校验
48     if (Array.isArray(homeData.recommend)) {
49         recommendList.value = homeData.recommend.filter(item => item.id &&
item.coverImg)
50     } else {
51         throw new Error('推荐列表数据缺失')
52     }
53 })
54 .catch((errorInfo) => {
55     // 分类处理异常
56     errorMsg.value = errorInfo.msg
57     // 权限错误特殊处理
58     if (errorInfo.type === 'permission') {
59         uniAsync.showModal({
60             title: '权限提示',
61             content: errorInfo.msg,
62             confirmText: '去设置'
63         }).then(() => uniAsync.openSetting())
64     }
65 })
66 .finally(() => {
67     // 无论成败都关闭加载
68     isLoading.value = false
69 })
70 }
71
72 // 页面加载时执行
73 loadHomeData()
74
75 // 重新加载数据
76 const reloadData = () => {
77     loadHomeData()
78 }
79 </script>

```

2. 详情页：多异步场景组合处理

详情页包含“获取萌宠信息”“点赞操作”“保存图片”三个核心异步操作，需分别处理各自的回调链路，避免相互干扰。

Code block

```

1  <script setup>
2  import { ref, onLoad } from 'vue'

```

```
3 import request from '../utils/request'
4 import { uniAsync, asyncUtil } from '../utils/asyncUtil'
5
6 const petId = ref('')
7 const petInfo = ref({})
8 const isLoading = ref(true)
9 const isLiking = ref(false) // 点赞请求状态锁
10
11 // 页面加载：获取萌宠详情
12 onLoad((options) => {
13   if (!options.id) {
14     uni.navigateBack()
15     return
16   }
17   petId.value = options.id
18   getPetDetail()
19 })
20
21 // 获取萌宠详情
22 const getPetDetail = () => {
23   isLoading.value = true
24   request.get(`/pet/${petId.value}`)
25     .then((detailData) => {
26       // 核心字段校验
27       if (!detailData.id || !detailData.name || !detailData.coverImg) {
28         throw new Error('萌宠信息不完整')
29       }
30       petInfo.value = detailData
31       // 顺带读取本地收藏状态
32       getCollectStatus()
33     })
34     .catch((errorInfo) => {
35       uni.showToast({ title: errorInfo.msg, icon: 'error' })
36       setTimeout(() => uni.navigateBack(), 1500)
37     })
38     .finally(() => {
39       isLoading.value = false
40     })
41 }
42
43 // 读取本地收藏状态
44 const getCollectStatus = () => {
45   uniAsync.getStorage(`collect_${petId.value}`)
46     .then((res) => {
47       petInfo.value.isCollected = res.data || false
48     })
49     .catch((err) => {
```

```

50      // 本地存储读取失败不阻断主流程，仅日志记录
51      console.error('读取收藏状态失败：', err)
52      petInfo.value.isCollected = false
53  })
54  }
55
56  // 点赞操作（防止重复点击）
57  const handleLike = () => {
58      if (isLiking.value) return
59      isLiking.value = true
60      request.post(`/pet/${petId.value}/like`, { isLike: !petInfo.value.isLiked })
61          .then((likeData) => {
62              petInfo.value.isLiked = likeData.isLiked
63              petInfo.value.likeCount = likeData.likeCount
64              uni.showToast({ title: likeData.isLiked ? '点赞成功' : '取消点赞' })
65          })
66          .catch((errorInfo) => {
67              uni.showToast({ title: errorInfo.msg, icon: 'error' })
68          })
69          .finally(() => {
70              isLiking.value = false
71          })
72  }
73
74  // 保存图片到相册
75  const savePetImage = () => {
76      if (!petInfo.value.coverImg) return
77      uniAsync.saveImageToPhotosAlbum(petInfo.value.coverImg)
78          .then(() => {
79              uni.showToast({ title: '图片保存成功' })
80              // 保存成功后更新本地保存记录
81              return uniAsync.setStorage(`saved_img_${petId.value}`, Date.now())
82          })
83          .catch((errorInfo) => {
84              if (errorInfo.type === 'permission') {
85                  uniAsync.showModal({
86                      title: '权限提示',
87                      content: '需要相册权限才能保存图片',
88                      confirmText: '去开启'

```

3. 图片工具类：原生API异步处理

图片预览中的长按保存、转发等原生功能，需通过Promise化封装后，应用完整的回调处理。

Code block

```

1  import { uniAsync, asyncUtil } from './asyncUtil'
2

```

```

3  export const imageUtil = {
4    /**
5     * 图片预览 (带长按操作)
6     * @param {Array} imgList - 图片URL数组
7     * @param {number} currentIndex - 当前预览索引
8     */
9    previewImage: (imgList, currentIndex = 0) => {
10     // 入参校验
11     if (!Array.isArray(imgList) || imgList.length === 0) {
12       uni.showToast({ title: '无预览图片', icon: 'none' })
13       return
14     }
15     const validIndex = Math.min(Math.max(currentIndex, 0), imgList.length - 1)
16     const currentImg = imgList[validIndex]
17
18     uni.previewImage({
19       urls: imgList.map(item => item.imgUrl || item),
20       current: validIndex,
21       longPressActions: {
22         itemList: ['保存图片', '转发图片'],
23         success: (res) => {
24           // 保存图片
25           if (res.tapIndex === 0) {
26             uniAsync.saveImageToPhotosAlbum(currentImg.imgUrl || currentImg)
27               .then(() => {
28                 uni.showToast({ title: '保存成功' })
29               })
30               .catch((errorInfo) => {
31                 console.error('保存图片异常: ', errorInfo)
32                 if (errorInfo.type === 'permission') {
33                   // 权限引导
34                   uniAsync.showModal({
35                     title: '权限不足',
36                     content: '请在设置中开启相册权限',
37                     confirmText: '去设置'
38                   }).then((modalRes) => {
39                     if (modalRes.confirm) uniAsync.openSetting()
40                   })
41                 }
42               })
43           }
44           // 转发图片
45           else if (res.tapIndex === 1) {
46             uniAsync.share({
47               title: `萌宠${currentImg.name || '图片'}`,
48               imageUrl: currentImg.imgUrl || currentImg,
49               path: `/pages/detail/detail?id=${currentImg.id || ''}`

```

```

50         })
51         .then(() => {
52             uni.showToast({ title: '转发成功' })
53         })
54         .catch((errorInfo) => {
55             uni.showToast({ title: errorInfo.msg, icon: 'error' })
56         })
57     }
58 }
59 }
60 })
61 }
62 }
63

```

4. 分类页：分页加载的异步状态管理

分类页的下拉刷新与上拉加载，需通过状态锁防止重复请求，在finally中重置状态，确保加载状态正确。

Code block

```

1  <script setup>
2  import { ref } from 'vue'
3  import request from '../utils/request'
4
5  const petList = ref([])
6  const page = ref(1)
7  const pageSize = ref(10)
8  const isRefreshing = ref(false)
9  const isLoadingMore = ref(false)
10 const noMoreData = ref(false)
11
12 // 下拉刷新
13 const onRefresh = () => {
14     isRefreshing.value = true
15     page.value = 1
16     noMoreData.value = false
17     loadPetList()
18 }
19
20 // 上拉加载更多
21 const onLoadMore = () => {
22     if (isLoadingMore.value || noMoreData.value) return
23     isLoadingMore.value = true
24     page.value += 1

```

```
25     loadPetList()
26   }
27
28   // 加载分类列表
29   const loadPetList = () => {
30     const params = { page: page.value, pageSize: pageSize.value }
31     request.get('/category/pet', params)
32       .then((listData) => {
33         // 校验列表数据格式
34         if (!Array.isArray(listData.list)) {
35           throw new Error('列表数据格式错误')
36         }
37         // 下拉刷新：覆盖数据；上拉加载：追加数据
38         if (page.value === 1) {
39           petList.value = listData.list
40         } else {
41           petList.value = [...petList.value, ...listData.list]
42         }
43         // 判断是否无更多数据
44         noMoreData.value = listData.list.length < pageSize.value
45       })
46       .catch((errorInfo) => {
47         // 上拉加载失败时回滚页码
48         if (page.value > 1) page.value -= 1
49         uni.showToast({ title: errorInfo.msg, icon: 'error' })
50       })
51       .finally(() => {
52         isRefreshing.value = false
53         isLoadingMore.value = false
54         // 停止下拉刷新动画
55         uni.stopPullDownRefresh()
56       })
57   }
58 </script>
```

四、进阶优化：异常处理与状态管理增强

1. 全局异常拦截

在App.vue中对未捕获的Promise异常进行全局拦截，避免页面崩溃，同时统一错误上报。

Code block

```
1 import { asyncUtil } from './utils/asyncUtil'
2
3 // 全局未捕获Promise异常监听
```

```

4  uni.onUncaughtPromiseError((err) => {
5    const errorInfo = asyncUtil.classifyError(err)
6    // 错误日志上报
7    console.error('[全局未捕获异常]', errorInfo.detail)
8    // 对用户可见的错误提示
9    if (errorInfo.type !== 'other') {
10     uni.showToast({ title: errorInfo.msg, icon: 'error' })
11   }
12 })
13
14 // App启动时初始化
15 onLaunch(() => {
16   // 初始化本地存储中的access-key
17   uniAsync.getStorage('access-key')
18     .then((res) => {
19     // 将密钥注入请求头
20     request.setAccessKey(res.data)
21   })
22   .catch(() => {
23     // 无密钥时跳转配置页
24     uni.navigateTo({ url: '/pages/setting/accessKey' })
25   })
26 })

```

2. 异步状态锁机制

对于点赞、收藏等高频交互，通过状态锁（如isLiking、isCollecting）防止重复触发异步请求，在finally中释放锁。

Code block

```

1  // 收藏操作（带状态锁）
2  const isCollecting = ref(false)
3  const handleCollect = () => {
4    if (isCollecting.value) return
5    isCollecting.value = true
6    const isCollect = !petInfo.value.isCollected
7    request.post(`/pet/${petId.value}/collect`, { isCollect })
8      .then((res) => {
9      petInfo.value.isCollected = res.isCollect
10     // 同步更新本地存储
11     return isCollect
12       ? uniAsync.setStorage(`collect_${petId.value}`, true)
13       : uniAsync.removeStorage(`collect_${petId.value}`)
14     })
15     .then(() => {

```

```
16         uni.showToast({ title: isCollect ? '收藏成功' : '取消收藏' })
17     })
18     .catch((errorInfo) => {
19         uni.showToast({ title: errorInfo.msg, icon: 'error' })
20     })
21     .finally(() => {
22         isCollecting.value = false
23     })
24 }
```

五、常见问题与解决方案

问题场景	错误原因	解决方案
then中直接使用数据导致崩溃	未校验数据格式，后端返回null或非预期类型	在then回调开头严格校验数据类型、核心字段，不满足则throw错误进入catch
加载动画一直显示	异步操作异常时未在catch中关闭加载状态	将关闭加载的逻辑统一放在finally中，确保必执行
重复请求导致数据混乱	高频操作（如快速点击点赞）触发多次异步请求	添加状态锁，请求发起时置为true，finally中置为false，锁为true时阻止新请求
权限错误用户无感知	catch中未区分异常类型，统一提示“请求失败”	使用异常分类工具，对权限错误单独处理，引导用户前往设置页

六、总结

本方案通过“基础封装-场景实现-进阶优化”的层级，为萌宠案例构建了完整的异步操作处理体系，核心价值体现在：

- 1. **数据安全**：then回调中的严格校验，从源头避免因数据异常导致的前端崩溃，提升页面稳定性。
- 2. **用户体验**：catch回调的分类提示，让用户清晰了解错误原因，权限错误等场景提供引导式解决方案。
- 3. **开发规范**：统一的异步处理范式，降低团队协作成本，便于后续维护与问题排查。
- 4. **状态可控**：finally回调与状态锁机制，确保页面状态始终可预测，避免加载动画卡死、重复请求等问题。

实际开发中，可结合项目规模进一步扩展，如引入Vuex/Pinia管理全局异步状态，或对接专业日志系统实现异常的可视化监控，让异步处理更高效、更可靠。

（注：文档部分内容可能由 AI 生成）